

HOST HEADER INJECTION

INTRODUCTION

In the world of web application security, headers play a critical role in managing communication between clients and servers. One such header is the Host header, which is essential for identifying the domain being requested. However, improper validation of this header can lead to a serious vulnerability known as Host Header Injection. This report explores the concept, impact, bypasses, and prevention techniques for this attack vector.

1. What is a Host Header?

The Host header is an HTTP request header used to specify the domain name of the server that the client wants to connect to. It is especially important in shared hosting environments where multiple domains are served from the same IP address.

Example:

GET / HTTP/1.1

Host: example.com

The server uses this header to determine which virtual host to respond with. This makes it a key component in routing and handling HTTP requests.

2. What is Host Header Injection?

Host Header Injection is a vulnerability that arises when the server trusts and reflects the Host header value from the HTTP request without proper validation. If an attacker can manipulate the Host header, they may be able to:

- Redirect users to malicious domains
- Bypass security controls
- Poison caches
- Conduct password reset poisoning attacks

- Perform web cache deception attacks

For example:

Host: attacker.com

If the application uses this unvalidated host in generating links, redirections, or password reset URLs, the attacker can hijack those processes.

3. What is the Impact of Host Header Injection?

The impact of Host Header Injection can vary from low to critical, depending on how the application handles the header. Some of the major risks include:

- Web Cache Poisoning: Alters the cache so future users receive attacker-controlled content
- Password Reset Poisoning: Malicious password reset links sent to users
- Open Redirects: Victims redirected to a malicious site
- Cross-site Scripting (XSS): In rare cases, injection into reflected HTML
- Internal IP disclosure or SSRF: By manipulating headers to route requests internally

4. How Do You Prevent It?

To prevent Host Header Injection, developers must ensure the application does not blindly trust user-supplied headers. Key prevention techniques include:

- Whitelist allowed domains: Only accept expected Host headers
- Use a canonical host: Override the header with a fixed, server-side value
- Avoid dynamic host usage: Don't use the Host header in logic like password reset URLs
- Web server configuration: Reject unexpected Host headers via Apache/Nginx settings
- Use modern frameworks: Many frameworks now have built-in protection if used properly

5. Bypasses for Host Header Injection

Attackers often use bypass techniques to manipulate host headers in creative ways:

1. X-Forwarded-Host Header
 - o Proxies may overwrite the host header with this.
 - o Example: *X-Forwarded-Host: attacker.com*
2. Multiple Host Headers
 - o Some servers parse the first or last instance:

Host: attacker.com

Host: victim.com

3. Injection with CRLF
 - o Used for response splitting:

Host: attacker.com%0d%0aInjected-Header: malicious

4. Using Port Numbers or Subdomains
 - o Example: *Host: victim.com.attacker.com* may trick validation logic
5. Via Proxy Misconfiguration
 - o Reverse proxies not properly stripping or validating Host headers

CONCLUSION

Host Header Injection is a lesser-known but highly dangerous vulnerability that, if exploited, can lead to a variety of attacks such as cache poisoning, redirection, and phishing. Through this task, I explored the core concept of host headers, how injection occurs, its critical impact, and the methods used by attackers to bypass basic filters. Proper validation and strict control over incoming headers are essential to protect applications from this type of exploitation.